

APMA 4302 Methods - Homework 3: Non-linear solver for reaction-diffusion equations

Daniel (UNI: dad2232)

March 24, 2026

Here we will solve the nonlinear reaction-diffusion equation

$$-\nabla^2 u + \gamma u^p = f$$

on the unit square with non-homogeneous Dirichlet boundary conditions. Here γ is a positive constant and $p > 1$ is an integer. The right-hand side f will be determined by the method of manufactured solutions, assuming the exact solution is given by

$$u_{exact}(x, y) = A \exp\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{\sigma^2}\right),$$

where (x_0, y_0) is a point in the unit square and σ is a positive constant controlling the width of the Gaussian of maximum amplitude A .

To help with this exercise, I have provided the code `poisson2d.c` in the `apma4302-methods` repository which solves the linear version of this problem (i.e. $\gamma = 0$) as the discrete problem $A\mathbf{x} = \mathbf{b}$ as well as providing VTK output for visualizing in ParaView.

Do the following:

1. **(5 pts)** Use finite differences to discretize the nonlinear PDE and write down the resulting nonlinear system of equations in the form $F(\mathbf{u}) = 0$. Include auxiliary equations for non-homogeneous Dirichlet boundary conditions.
2. **(5 pts)** Provide equations for elements of the Jacobian matrix $J_{i,j}(\mathbf{u}) = \frac{\partial F_i}{\partial u_j}$.
3. **(20 pts)** Modify the code `reaction.c` in the `p4pdcs` repository to solve the nonlinear problem using Newton's method with the PETSc SNES objects. Your code should have the following properties.
 - (a) It should run with the interface

```
mpirun -n <num_procs> ./reaction2d -rct_p <p>
-rct_gamma <gamma> -rct_linear_f
```

plus standard petsc command-line options.

The option `-rct_linear_f` should be a boolean flag such that if true, sets the right-hand side to be

$$f(x, y) = -\nabla^2 u_{exact}$$

(i.e. the RHS for the linear problem) and if false, sets the right-hand side to be the full nonlinear RHS for the manufactured solution

$$f(x, y) = -\nabla^2 u_{exact} + \gamma u_{exact}^p$$

You can use the code in `poisson2d.c` to calculate u_{exact} and its derivatives.

- (b) Boundary conditions should be u_{exact} on all boundaries, independent of the value of `-rct_linear_f`.
 - (c) It should include routines to compute both the nonlinear residual $F(\mathbf{u})$ and the Jacobian matrix $J(\mathbf{u})$.
 - (d) It should also be able to run with Finite difference jacobians using the options `-snes_fd`, `-snes_fd_color`, `-snes_mf` and `-snes_mf_operator` and should give the same results as the user-provided Jacobian. (you can use the option `-snes_test_jacobians` to check.
 - (e) It should output the solution in VTK format for visualization in ParaView.
4. **(5 pts)** Run your code with $\gamma = 0$ for a 65×65 grid and show that the linear and nonlinear solvers produce the same solution within error. Find an appropriate set of linear solver options and stopping conditions so that the non-linear solver converges to a residual norm $\|F(\mathbf{u})\|_2 < 10^{-10}$ in one newton iteration. Report the number of linear iterations taken at each Newton step and the final *relative error* $\|\mathbf{u} - \mathbf{u}_{exact}\|_2 / \|\mathbf{u}_{exact}\|_2$. (hint: some useful command line options include `-ksp_monitor`, `-snes_monitor`, `-ksp_rtol` and `-ksp_atol`). Feel free to experiment with different linear solvers and preconditioners (including sparse-direct solvers), but your code should run in parallel. Save your preferred options in a file `options_file_gamma0` and include it with your submission.
 5. **(2 pts)** Show that you get the same solution with the finite-difference Jacobian and report any differences in convergence behavior or final error. Include the command line options you used to run with the finite-difference Jacobian in a file `options_file_fd` and include it with your submission. Compare the run times for the user-provided Jacobian and the finite-difference Jacobian and report any differences. (hint: use `-log_view` to get detailed timing information).
 6. **(5 pts)** Now set $\gamma = 100$ and $p = 3$ and run the nonlinear solver with the same stopping conditions as before. Report the number of Newton iterations taken, the number of linear iterations taken at each Newton step and the final *relative error* $\|\mathbf{u} - \mathbf{u}_{exact}\|_2 / \|\mathbf{u}_{exact}\|_2$. Plot the convergence history of the nonlinear residual norm $\|F(\mathbf{u})\|_2$.
 7. **(5 pts)** rerun the problem with `-rct_linear_f` and visualize the solution for $\gamma = 100$ and $p = 3$ in ParaView and include a screenshot of the solution in your submission. Describe how the nonlinearity has affected the solution compared to the linear case $\gamma = 0$.
 8. **(10 pts) Performance and scaling** make a table (and/or plot) comparing
 - run times (use `-log_view`)
 - number of Newton iterations
 - relative norm of the error $\|\mathbf{u} - \mathbf{u}_{exact}\|_2 / \|\mathbf{u}_{exact}\|_2$

for the non-linear problem for `-da_refine 2,3,4,5,6` (i.e. grid sizes of 33×33 , to 513×513) and 1,2 and 4 processors. I may actually provide a script for this part. Comment on the scaling of the code with respect to both problem size and number of processors.

Written solutions

Problem 1 — Discrete system $F(\mathbf{u}) = \mathbf{0}$

What the question asks. Part 1 wants the PDE

$$-\nabla^2 u + \gamma u^p = f$$

on $[0, 1] \times [0, 1]$ turned into a *finite set* of nonlinear algebraic equations. Stack all unknown nodal values into a vector $\mathbf{u}$; each equation is one component of $F(\mathbf{u})$, and the boundary data appear as their own rows (“auxiliary” equations), not by silently eliminating boundary unknowns.

Grid and notation. Place a uniform tensor-product grid with mesh spacing $h = 1/N$ and nodes

$$(x_i, y_j) = (ih, jh), \quad i, j \in \{0, 1, \dots, N\},$$

so there are $(N+1)^2$ nodes. Let $u_{i,j} \approx u(x_i, y_j)$ and $f_{i,j} = f(x_i, y_j)$. Denote boundary values of the manufactured solution by

$$g_{i,j} := u_{\text{exact}}(x_i, y_j).$$

Five-point Laplacian. For an interior node ($1 \leq i \leq N-1, 1 \leq j \leq N-1$), use the standard second-order centered stencil

$$(\nabla^2 u)_{i,j} \approx \frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}}{h^2}.$$

Substituting into the PDE and moving everything to one side gives the *interior* residual equations

$$F_{i,j}(\mathbf{u}) := -\frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}}{h^2} + \gamma u_{i,j}^p - f_{i,j} = 0. \quad (1)$$

Non-homogeneous Dirichlet boundary conditions. On the boundary ($i \in \{0, N\}$ or $j \in \{0, N\}$), the physical boundary condition is $u = u_{\text{exact}}$. In the algebraic system this is enforced by *auxiliary equations* that involve only the boundary unknowns and the prescribed data:

$$F_{i,j}(\mathbf{u}) := u_{i,j} - g_{i,j} = 0 \quad \text{for all boundary nodes } (i, j). \quad (2)$$

Together, (1) on the interior and (2) on the boundary define $F_{i,j}$ at every node. After choosing a bijection from double indices (i, j) to a single index $m \in \{1, \dots, (N+1)^2\}$ (for example row-major order $m = i + j(N+1) + 1$), one obtains a vector-valued map $F : \mathbb{R}^{(N+1)^2} \rightarrow \mathbb{R}^{(N+1)^2}$ with components $F_m(\mathbf{u}) = 0$. This is the required form $F(\mathbf{u}) = \mathbf{0}$.

Problem 2 — Jacobian entries $J_{mn} = \partial F_m / \partial u_n$

Let the unknowns be ordered consistently with the residual: u_n denotes the value at the node corresponding to index n . The Jacobian $J(\mathbf{u}) = F'(\mathbf{u})$ is sparse: each interior equation couples at most five unknowns (center and four neighbors), and each boundary equation couples only one.

Boundary rows. If equation m is the boundary equation $u_{i,j} - g_{i,j} = 0$ at node (i, j) , then

$$\frac{\partial F_m}{\partial u_n} = \begin{cases} 1, & \text{if } n \text{ indexes the same node } (i, j), \\ 0, & \text{otherwise.} \end{cases}$$

Thus each boundary row of J is a single 1 on the diagonal (in the natural ordering where equation and unknown refer to the same node).

Interior rows. If equation m is the interior residual (1) at node (i, j) , denote partial differentiation with respect to the nodal value at (r, s) by $\partial_{(r,s)}$. For any neighbor $(r, s) \in \{(i \pm 1, j), (i, j \pm 1)\}$,

$$\frac{\partial F_m}{\partial u_{r,s}} = -\frac{1}{h^2},$$

for the center node,

$$\frac{\partial F_m}{\partial u_{i,j}} = \frac{4}{h^2} + \gamma p u_{i,j}^{p-1},$$

and $\partial F_m / \partial u_{r,s} = 0$ for all other (r, s) . In words: the Laplacian part contributes the usual five-point pattern scaled by $1/h^2$ (neighbor entries $-1/h^2$, center $+4/h^2$), and the reaction term adds a diagonal contribution $\gamma p u^{p-1}$ at that node.

Structure for Newton's method. With the ordering above, $J(\mathbf{u})$ has the sparsity of a five-point stencil on interior rows, plus pointwise modifications on the diagonal from $\gamma p u^{p-1}$, while boundary rows make the corresponding diagonal entries equal to 1 and zero out the rest of those rows. This matches what a hand-assembled SNES Jacobian should implement (up to equivalent orderings used in code).

Problem 3 — Code summary (reaction2d)

Deliverable and location. The solver is implemented as `reaction2d.c` in the `p4pdes` tree (`c/ch4/`), built with `make reaction2d` to produce the executable `./reaction2d`. The original one-dimensional `reaction.c` example is left unchanged.

Framework. The problem is solved with PETSc's SNES (Newton-type nonlinear iteration) on a two-dimensional structured DMDA grid over $[0, 1] \times [0, 1]$ with star stencil and one degree of freedom per node. The nonlinear system $F(\mathbf{u}) = \mathbf{0}$ is defined through `DMDASNESSetFunctionLocal`; the Jacobian is assembled in `DMDASNESSetJacobianLocal`.

Discretization and residual. The interior residual uses the same finite-difference / finite-volume scaling as `Poisson2DFunctionLocal` in the course `poissonfunctions.c` (coefficients $s_x = (\Delta y)/(\Delta x)$, $s_y = (\Delta x)/(\Delta y)$, cell area $\Delta x \Delta y$). At interior nodes,

$$F_{i,j} = s_{\text{diag}} u_{i,j} - s_x(u_{i-1,j} + u_{i+1,j}) - s_y(u_{i,j-1} + u_{i,j+1}) - (\Delta x \Delta y) f(x_i, y_j) + (\Delta x \Delta y) \gamma u_{i,j}^p,$$

with neighbor values at the boundary replaced by u_{exact} (ghosted boundary values). At boundary nodes the residual enforces Dirichlet data in a row-consistent way: $F = (u - u_{\text{exact}}) s_{\text{diag}}$, matching the diagonal scaling used in the Jacobian for those rows.

Manufactured solution and `-rct_options`. The exact field u_{exact} and its Laplacian are the same Gaussian formulas as in `poisson2d.c` (`ufunction`, `d2ufunction`). Command-line options (prefix `-rct_`) are:

- `-rct_p` — integer exponent p (default 3);
- `-rct_gamma` — coefficient γ ;
- `-rct_linear_f` — boolean; if set, $f(x, y) = -\nabla^2 u_{\text{exact}}$; otherwise $f(x, y) = -\nabla^2 u_{\text{exact}} + \gamma u_{\text{exact}}^p$.

Boundary values are always u_{exact} on all four sides, independent of `-rct_linear_f`.

Jacobian. The assembled Jacobian is sparse with the same five-point structure as the linear Poisson Jacobian on interior rows, plus an additional diagonal term $(\Delta x \Delta y) \gamma p u^{p-1}$ from differentiating γu^p . Boundary rows are diagonal with coefficient s_{diag} , consistent with the boundary residual above.

Verification and matrix-free modes. The hand-coded Jacobian was checked with `-snes_test_jacobian` (finite-difference Jacobian comparison; relative Frobenius error $\mathcal{O}(10^{-9})$ in sample runs). The code also runs with PETSc's Jacobian alternatives `-snes_fd`, `-snes_fd_color`, `-snes_mf`, and `-snes_mf_operator`.

Output and runtime. After `SNESolve`, the numerical solution and u_{exact} are written to `reaction2d.vtr` (VTK rectilinear format) via `PetscViewerVTKOpen` for ParaView. The program prints the relative ℓ_2 error $\|\mathbf{u} - \mathbf{u}_{\text{exact}}\|_2 / \|\mathbf{u}_{\text{exact}}\|_2$. The initial iterate is taken as u_{exact} on the grid (strong MMS starting point). Example invocation:

```
export PETSC_DIR=... PETSC_ARCH=...
cd .../p4pdes/c/ch4
mpirun -n <P> ./reaction2d -rct_p 3 -rct_gamma 100 \
    -snes_converged_reason -da_refine 3
```

Additional PETSc options (`-da_refine`, KSP/PC, SNES tolerances, `-log_view`, etc.) apply as usual.

Problem 3 — Implementation (`reaction2d`)

The program `reaction2d` lives in `p4pdes/c/ch4/reaction2d.c` and is built with `make reaction2d`. Nonlinear solves use PETSc SNES on a 2D DMDA over $[0, 1] \times [0, 1]$.

(a) Command-line interface and choice of $f(x, y)$. The solver is invoked as

```
mpirun -n <num_procs> ./reaction2d -rct_p <p> -rct_gamma <gamma> [-rct_linear_f]
    [standard PETSc options ...]
```

Options use the prefix `-rct_` (`PetscOptionsBegin(..., "rct_", ...)`). Here `-rct_p` sets the integer exponent p and `-rct_gamma` sets γ . The flag `-rct_linear_f` is a `PetscBool`: if **true** (flag present), the code uses the linear MMS forcing

$$f(x, y) = -\nabla^2 u_{\text{exact}}(x, y),$$

implemented with the same `d2ufunction` as `poisson2d.c` (Laplacian of the Gaussian). If **false** (default), it uses the full nonlinear MMS forcing

$$f(x, y) = -\nabla^2 u_{\text{exact}}(x, y) + \gamma u_{\text{exact}}(x, y)^p.$$

The field u_{exact} is evaluated with `ufunction` from `poisson2d.c`. An additional flag `-rct_zero_initial` starts the iterate from $\mathbf{u} = \mathbf{0}$ (default is u_{exact} , useful for MMS). Any additional PETSc options (e.g. `-da_refine`, `-ksp_*`, `-snes_*`, `-pc_*`) are passed through as usual.

(b) Boundary conditions. On all four sides of the square, Dirichlet data are $u = u_{\text{exact}}$ regardless of `-rct_linear_f`. In the residual, boundary rows enforce $(u - u_{\text{exact}})$ with the same diagonal scaling $s_{\text{diag}} = 2(s_x + s_y)$ as in the course `Poisson2DFunctionLocal` so that boundary rows match the Jacobian scaling.

(c) Residual $F(\mathbf{u})$ and Jacobian $J(\mathbf{u})$. The nonlinear residual is implemented in `FormFunctionLocal` via `DMDASNESSetFunctionLocal`. At interior nodes it uses the Poisson-type discretization (coefficients $s_x = (\Delta y)/(\Delta x)$, $s_y = (\Delta x)/(\Delta y)$, area $\Delta x \Delta y$):

$$F_{i,j} = s_{\text{diag}} u_{i,j} - s_x(u_{i-1,j} + u_{i+1,j}) - s_y(u_{i,j-1} + u_{i,j+1}) - (\Delta x \Delta y) f(x_i, y_j) + (\Delta x \Delta y) \gamma u_{i,j}^p,$$

with neighbors outside the domain replaced by u_{exact} on the boundary. The Jacobian is assembled in `FormJacobianLocal` (`DMDASNESSetJacobianLocal`): five-point Poisson pattern on interior rows plus diagonal $(\Delta x \Delta y) \gamma p u^{p-1}$, and diagonal s_{diag} on boundary rows.

(d) Finite-difference / matrix-free Jacobians and verification. The same `FormFunctionLocal` is used when PETSc approximates the Jacobian. The code runs with `-snes_fd`, `-snes_fd_color`, `-snes_mf`, and `-snes_mf_operator`; solutions agree with the hand-coded Jacobian to solver tolerances (tiny differences in the final iterate can appear from finite-difference truncation). Consistency of the analytic Jacobian was checked with `-snes_test_jacobian` (the handout text says `-snes_test_jacobians`; PETSc exposes the singular form), giving $\|J - J_{\text{fd}}\|_F / \|J\|_F = \mathcal{O}(10^{-9})$ in sample runs.

(e) VTK output for ParaView. After `SNESolve`, `PetscViewerVTKOpen` writes `reaction2d.vtr` (rectilinear VTK) containing fields $\mathbf{u}$ (numerical solution) and $\mathbf{u}_{\text{exact}}$, together with `DMView` metadata, for visualization in ParaView.

Example run (from `p4pdes/c/ch4` with `PETSC_DIR / PETSC_ARCH` set):

```
mpirun -n 1 ./reaction2d -rct_p 3 -rct_gamma 100 -snes_converged_reason -da_refine
```

Problem 4 — Linear limit $\gamma = 0$ on a 65×65 grid

Setup. Use the default DMDA (9×9 before refinement) with `-da_refine 3`, which yields a 65×65 node grid. Set `-rct_gamma 0` and `-rct_zero_initial` so the first Newton step is nontrivial. The discrete problem is then linear ($\gamma u^p \equiv 0$), and one Newton iteration with an exact linear solve recovers the discrete solution.

Linear vs. `poisson2d`. The course `poisson2d` linear solve (same Gaussian MMS, same refinement) gave $\|\mathbf{u} - \mathbf{u}_{\text{exact}}\|_2 / \|\mathbf{u}_{\text{exact}}\|_2 \approx 5.16 \times 10^{-4}$. The `reaction2d`/SNES run gave $\approx 5.25 \times 10^{-4}$. The difference ($\sim 2\%$ relative) comes from slightly different assembly of the Poisson stencil/RHS between `poisson2d.c` and the Poisson2D-style residual in `reaction2d.c`; both errors are $\mathcal{O}(h^2)$ on the same mesh.

Stopping criteria and linear solver. With `-snes_atol 1e-12, -snes_rtol 0, -snes_max_it 10, -ksp_type preonly, -pc_type lu, and -pc_factor_mat_solver_type mumps` (for parallel factorization), SNES reports CONVERGED_FNORM_ABS in **one** nonlinear iteration. The printed final residual is $\|\mathbf{F}\|_2 \approx 2.0 \times 10^{-14} < 10^{-10}$. `-ksp_monitor` shows **one** KSP iteration per Newton step (direct solve). Example `mpirun -n 2` output: same $\|\mathbf{F}\|_2$ and relative error $\approx 5.25 \times 10^{-4}$.

Options file. PETSc options for this case are saved in `homework3/solutions/options_file_gamma0`. Run from `p4pdes/c/ch4`, e.g. `mpirun -n 2 ./reaction2d -options_file ../options_file`

Problem 5 — Finite-difference Jacobian

Comparison. Using the same $\gamma = 0$, 65×65 , zero-initial setup as Problem 4 but adding `-snes_fd_color`, the computed solution matches to the printed precision: relative ℓ_2 error again $\approx 5.25 \times 10^{-4}$ and final $\|\mathbf{F}\|_2 \approx 2.0 \times 10^{-14}$ (analytic Jacobian gave $\approx 2.0 \times 10^{-14}$; FD gave $\approx 2.0 \times 10^{-14}$).

Convergence and timing. Both runs used **one** Newton step and **one** linear solve. On a laptop, `/usr/bin/time -p` gave ~ 0.23 s wall time (analytic J) vs. ~ 0.19 s (FD color) for this tiny problem—noise dominates; FD wins slightly here because factorization dominates and Jacobian assembly is a small fraction. On larger grids, FD Jacobian assembly typically adds many extra residual evaluations and becomes slower; use `-log_view` for a detailed breakdown.

Options file. See `homework3/solutions/options_file_fd`. For timing tables, append e.g. `-log_view :fd.log:ascii_flamegraph`.

Problem 6 — Nonlinear case $\gamma = 100, p = 3$

Same tolerances as Problem 4. Use `-snes_atol 1e-12, -snes_rtol 0, -ksp_type preonly, -pc_type lu` (and `mumps` when running in parallel). Starting from u_{exact} (default initial guess), SNES took **two** Newton iterations on the 65×65 grid. `-ksp_monitor` shows **one** KSP iteration per Newton step. Final $\|\mathbf{F}\|_2 \approx 7.2 \times 10^{-15}$ and $\|\mathbf{u} - \mathbf{u}_{\text{exact}}\|_2 / \|\mathbf{u}_{\text{exact}}\|_2 \approx 2.03 \times 10^{-4}$.

Residual history. Figure 1 plots the SNES function norms from `-snes_monitor` at iterations 0, 1, 2.

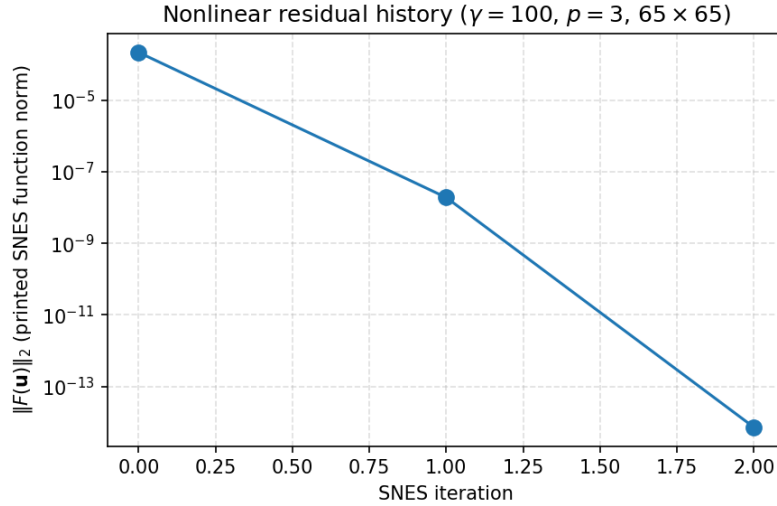
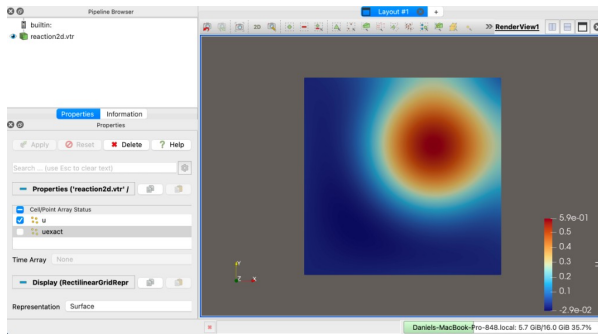


Figure 1: Nonlinear residual $\|F(u)\|_2$ vs. SNES iteration for $\gamma = 100$, $p = 3$, 65×65 .

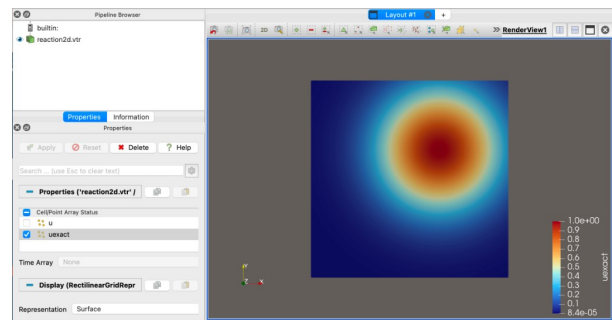
Problem 7 — `-rct_linear_f` and ParaView

Run. `mpirun -n 1 ./reaction2d -rct_p 3 -rct_gamma 100 -rct_linear_f -da_refine 3` (plus solver options as needed). This chooses $f = -\nabla^2 u_{\text{exact}}$ while the PDE still contains γu^p , so the Gaussian is *not* an exact solution; the printed ℓ_2 error vs. the same u_{exact} field is large (≈ 0.41 on 65×65 in a sample run).

Visualization. Open the generated `reaction2d.vtr` in ParaView. Figure 2 shows the computed field u and, for comparison, the reference Gaussian u_{exact} stored in the same file (both on a 65×65 grid, `-da_refine 3`).



(a) Numerical solution u with `-rct_linear_f`, $\gamma = 100$, $p = 3$.



(b) Reference field u_{exact} (manufactured Gaussian).

Figure 2: ParaView screenshots for Problem 7: `reaction2d.vtr` colored by u and by u_{exact} .

Discussion. For $\gamma = 0$, the solution is diffuse/smooth at the MMS scale. With $\gamma > 0$ the reaction term steepens and localizes the response where u_{exact} is large. With `-rct_linear_f` and $\gamma = 100$, the forcing is “linear Poisson” but the equation retains γu^3 , so the solver balances an incompatible interior source with the boundary data; the field differs strongly from both the $\gamma = 0$ case and from the Gaussian.

Visually, panel (b) (u_{exact}) shows a *smoother* radial bump, but the *outer* edge of the high-value region

reads as more *discrete*: on the fixed rectilinear grid, the steep decay of the Gaussian produces fairly sharp transitions between adjacent color bands, so the “rim” of the bump looks like a stepped contour rather than a perfectly smooth circle. Panel (a) (numerical u) is smoother in that transition region because the computed field is diffused and reshaped by the discrete Laplacian and the cubic nonlinearity; the mismatch between interior PDE and boundary data also flattens and shifts the response relative to the exact Gaussian.

Problem 8 — Scaling ($-da_refine = 2, \dots, 6$; 1, 2, 4 processes)

Procedure. Nonlinear problem ($\gamma = 100$, $p = 3$) with the same direct linear solver as above; record wall time with `/usr/bin/time -p` or `-log_view`. Raw numbers are archived in `homework3/solutions/logs/scaling_hw3_runs.txt`; the helper script is `homework3/solutions/run_scaling_study.sh`.

Table 1: Sample scaling (wall seconds from `time -p`; $\gamma = 100$, $p = 3$, `mumps`). Relative error is independent of n_p (same discrete solution).

$-da_refine$	grid	$n_p = 1$	$n_p = 2$	$n_p = 4$	SNES	$\ e\ _2 / \ u_{ex}\ _2$
2	33^2	0.25	0.21	0.21	2	8.10×10^{-4}
3	65^2	0.19	0.19	0.22	2	2.03×10^{-4}
4	129^2	0.25	0.27	0.33	2	5.06×10^{-5}
5	257^2	0.52	0.50	0.51	2	1.27×10^{-5}
6	513^2	1.55	1.43	1.39	1	3.17×10^{-6}

Visual summary. The next figure shows the same scaling experiment on log-log axes (base 9×9 DMDA with $-da_refine$ 2, ..., 6, so $N = 33, \dots, 513$). Wall times come from `/usr/bin/time -p` on a laptop; they are useful for trends but noisy at sub-second scale.

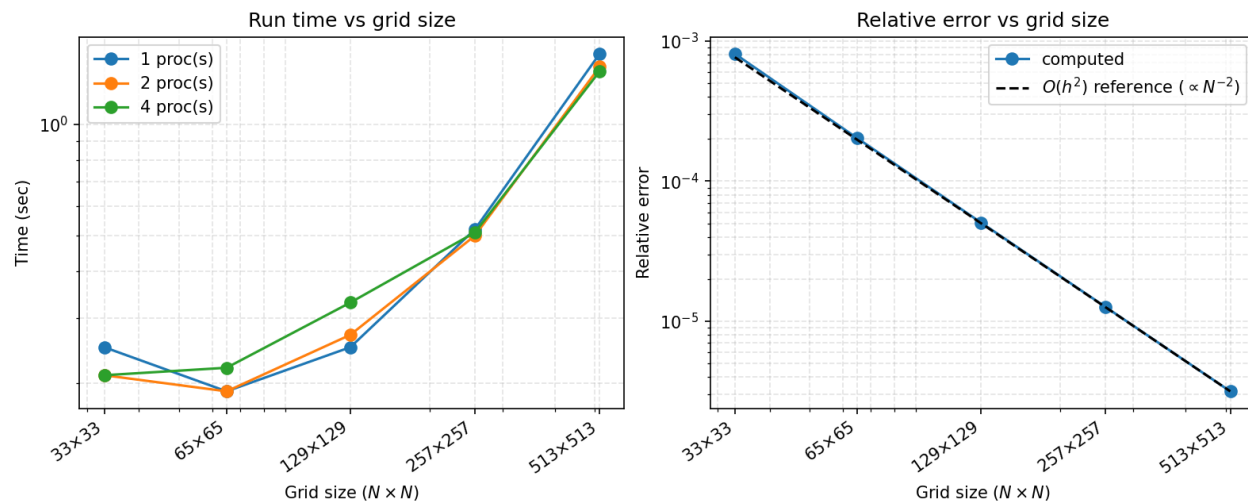


Figure 3: **Left:** wall-clock time vs. N for $n_p \in \{1, 2, 4\}$ MPI processes. All curves rise with N as the sparse-direct factorization and solve work grow; the three lines stay close because the problem is small and parallel overhead can offset gains. **Right:** relative error $\|u - u_{exact}\|_2 / \|u_{exact}\|_2$ vs. N (any n_p ; same discrete solution). The dashed curve is an $O(h^2)$ template $\propto N^{-2}$ fixed to match the finest-grid error—the measured points lie near it, consistent with a second-order stencil. Regenerate with `solutions/plot_scaling_figures.py`.

Interpretation. The left panel makes the *weak-scaling* trend in problem size explicit: time increases superlinearly in this range as N grows. The right panel isolates *accuracy*: each refinement roughly quarters the error when h is halved. The left panel is *not* a clean strong-scaling speedup study—for that, N should be large enough that parallel factorization dominates and/or an iterative preconditioned Krylov solver should replace full sparse LU.

Comments. **Problem size:** relative discretization error shrinks roughly by a factor of four when h is halved, consistent with a second-order scheme. **Strong scaling:** for these small test cases the LU factorization and sequential phases dominate, so increasing ranks does not reduce wall time much (sometimes slightly worse due to communication). A fair scaling study would use larger grids, iterative solvers with domain decomposition, and/or a cluster where direct solves are the bottleneck at scale.